Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

Кафедра оптоелектроніки та інформаційних технологій

Звіт

до лабораторної роботи №4

Варіант 11

на тему

«Рекурсія та мемоізація в Python»

Виконала:
Студентка групи ФЕП-21
Муковоз М. Т.

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета:

- ознайомитися з рекурсивним підходом до розв’язання задач та особливостями рекурсії в Python;
- реалізувати рекурсивні алгоритми та обходи бінарного дерева;
- застосувати мемоізацію за допомогою functools.lru_cache;
- порівняти рекурсивні, мемоізовані та ітеративні реалізації за часом виконання.

Теоретичні відомості

Рекурсія — це спосіб розв’язання задачі, за якого функція викликає сама себе для обробки простішого випадку тієї самої задачі. Рекурсивна функція повинна містити базовий випадок, який зупиняє подальші виклики, та рекурсивний крок, що поступово наближає виконання до базового випадку.

Під час кожного рекурсивного виклику Python зберігає інформацію про поточний стан функції у стеку викликів. Глибина рекурсії обмежена системним лімітом інтерпретатора. У CPython оптимізація хвостової рекурсії (tail call optimization) не реалізована, тому навіть хвостові рекурсивні виклики продовжують використовувати стек.

Мемоізація — це збереження результатів уже виконаних обчислень, щоб не повторювати їх надалі. У Python для цього можна використовувати декоратор functools.lru_cache. Він кешує результати викликів функції з однаковими аргументами. Переглянути статистику кешу можна за допомогою cache_info(), а очистити його — cache_clear().

Ітеративний алгоритм використовує цикли замість рекурсивних викликів. Для задач із великою кількістю повторних підзадач ітеративний підхід часто дозволяє зменшити витрати пам’яті та уникнути обмеження глибини рекурсії.

Хід роботи

1. Було створено файл algorithms.py, у якому реалізовано три способи розв’язання задачі варіанта 11 — підрахунку кількості способів піднятися на сходи. За один крок можна подолати 1, 2 або 3 сходинки. Для n сходинок кількість способів визначається рекурентною формулою: $f(n) = f(n - 1) + f(n - 2) + f(n - 3)$. Базові випадки: $f(0) = 1$, а для $n < 0$ результат дорівнює 0.

Реалізовано функцію stairs_naive() без кешування, функцію stairs_memo() з декоратором @lru_cache(maxsize=None), а також ітеративну функцію stairs_iter().

Реалізація задачі про сходи у файлі algorithms.py

algorithms.py > check_n

```python

from __future__ import annotations

from collections import deque
from dataclasses import dataclass
from functools import lru_cache
from random import choice
from typing import Generator, Optional


def check_n(n: int) -> None:
    if not isinstance(n, int) or isinstance(n, bool):
        raise TypeError("n must be an integer")


def stairs_naive(n: int) -> int:
    check_n(n)

    if n < 0:
        return 0
    if n == 0:
        return 1

    return (
        stairs_naive(n - 1)
        + stairs_naive(n - 2)
        + stairs_naive(n - 3)
    )


@lru_cache(maxsize=None)
def stairs_memo(n: int) -> int:
    check_n(n)

    if n < 0:
        return 0
    if n == 0:
        return 1

    return (
        stairs_memo(n - 1)
        + stairs_memo(n - 2)
        + stairs_memo(n - 3)
    )


def stairs_iter(n: int) -> int:
    check_n(n)

    if n < 0:
        return 0
    if n == 0:
        return 1
    if n == 1:
        return 1
    if n == 2:
        return 2

    a, b, c = 1, 1, 2

    for _ in range(3, n + 1):
        a, b, c = b, c, a + b + c

    return c
```

2. Для демонстрації рекурсивних алгоритмів також реалізовано швидке сортування quicksort() з випадковим вибором опорного елемента. Функція повертає новий відсортований список і не змінює вхідний список.

Реалізація quicksort()

```python
def quicksort(xs: list[int]) -> list[int]:
    if len(xs) <= 1:
        return xs[:]

    pivot = choice(xs)

    left = [x for x in xs if x < pivot]
    middle = [x for x in xs if x == pivot]
    right = [x for x in xs if x > pivot]

```

3. Було створено клас Node для представлення вузлів бінарного дерева. Для дерева реалізовано рекурсивні обходи preorder(), inorder() і postorder(), а також ітеративні обходи bfs_level_order() та dfs_preorder_iter().

Клас Node та функції обходу бінарного дерева

```python
@dataclass(slots=True)
class Node:
    key: int
    left: Optional[Node] = None
    right: Optional[Node] = None


def preorder(t: Optional[Node]) -> Generator[int, None, None]:
    if t is None:
        return

    yield t.key
    yield from preorder(t.left)
    yield from preorder(t.right)


def inorder(t: Optional[Node]) -> Generator[int, None, None]:
    if t is None:
        return

    yield from inorder(t.left)
    yield t.key
    yield from inorder(t.right)


def postorder(t: Optional[Node]) -> Generator[int, None, None]:
    if t is None:
        return

    yield from postorder(t.left)
    yield from postorder(t.right)
    yield t.key

```

```python
def bfs_level_order(t: Optional[Node]) -> list[int]:
    if t is None:
        return []

    queue = deque([t])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node.key)

        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)

    return result


def dfs_preorder_iter(t: Optional[Node]) -> list[int]:
    if t is None:
        return []

    stack = [t]
    result = []

    while stack:
        node = stack.pop()
        result.append(node.key)

        if node.right:
            stack.append(node.right)

        if node.left:
            stack.append(node.left)

    return result
```

4. Для додаткового прикладу мемоізації реалізовано функцію fib() для обчислення чисел Фібоначчі з використанням @lru_cache, а також ітеративну функцію fib_iter(). Для перевірки роботи кешу використано методи cache_info() і cache_clear().

Реалізація функцій fib() та fib_iter()

```python

@lru_cache(maxsize=None)
def fib(n: int) -> int:
    check_n(n)

    if n < 0:
        raise ValueError("n must be non-negative")

    if n < 2:
        return n

    return fib(n - 1) + fib(n - 2)

```

```python
def fib_iter(n: int) -> int:
    check_n(n)

    if n < 0:
        raise ValueError("n must be non-negative")

    a, b = 0, 1

    for _ in range(n):
        a, b = b, a + b

    return a

```

5. Для перевірки правильності роботи програми створено автоматизовані тести за допомогою pytest. Тести перевіряють базові та граничні випадки задачі про сходи, відповідність рекурсивної, мемоізованої та ітеративної реалізацій, роботу кешу, швидке сортування, обходи дерева та обчислення чисел Фібоначчі.

Успішне виконання тестів pytest

```
(.venv) C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_04>python -m pytest -q
..................
18 passed in 0.11s
```

6. За допомогою файлу benchmark.py проведено вимірювання часу виконання трьох реалізацій задачі про сходи. Для порівняння використано різні значення n. Перед вимірюванням мемоізованої функції кеш очищувався, щоб результати були коректними та не залежали від попередніх запусків.

Результати порівняння часу виконання benchmark.py

```
(.venv) C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_04>python benchmark.py
n        Naive (s)        Memo (s)         Iterative (s)
10       0.00029330       0.00001940       0.00000800
20       0.08308380       0.00005070       0.00000950
25       1.72619540       0.00005380       0.00000960
```

7. Підготовлено конфігураційні файли проєкту: requirements.txt, pyproject.toml, mypy.ini та .gitignore. У README.md описано призначення програми, принцип роботи рекурсії та мемоізації, складність реалізацій і команди для запуску тестів та перевірок.

Висновок

У ході виконання лабораторної роботи №4 за варіантом 11 було досліджено рекурсію та мемоізацію в Python. Для задачі підрахунку кількості способів піднятися на сходи з кроками 1, 2 або 3 реалізовано наївний рекурсивний,

мемоізований та ітеративний алгоритми. Порівняння показало, що наївна рекурсія повторно обчислює однакові підзадачі, тоді як мемоізація зберігає вже отримані результати та зменшує кількість обчислень. Ітеративна реалізація також має лінійну часову складність і не використовує стек рекурсивних викликів.

Крім цього, реалізовано швидке сортування, рекурсивні та ітеративні обходи бінарного дерева, а також рекурсивну й ітеративну реалізації обчислення чисел Фібоначчі. Правильність роботи функцій перевірено автоматизованими тестами pytest, а швидкодію алгоритмів досліджено за допомогою benchmark.py. Під час роботи було закріплено розуміння базових випадків рекурсії, обмеження глибини рекурсивних викликів у Python та переваг використання мемоізації.